LibTorch : Day 5

Understanding AutoGrads

Autograds are super important as these are the automatic differentiation tool of PyTorch / LibTorch.

* Why do we need it ?

Suppose, we have mathematical relationship →

$y = x^2$ ⟶ program ⟶ (x)

(suppose we write a program) → x → $\frac{\partial y}{\partial x}$

$\frac{dy}{dx} = 2x$ ⟶ We can formulate all of this into a code.

But what if complexity is increased ?

suppose We do, $y = x^2$

$z = \sin(y)$ → $x \rightarrow \frac{dz}{dx}$

This scenario is more complex compared to previous ones. → because nested functions involved

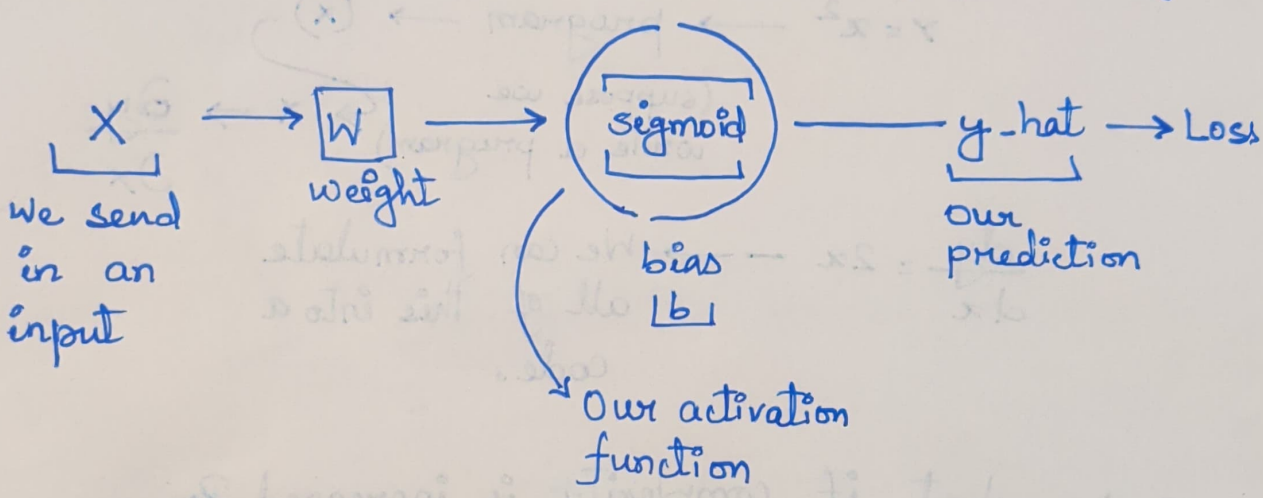
So, we find out $\frac{dz}{dy} = \cos y$ and $\frac{dy}{dx} = 2x$

$$\frac{dz}{dx} = \frac{dz}{dy}\,\frac{dy}{dx}$$

$$\hookrightarrow 2x\cos(y) = 2x\cos(x^2)$$

As we handle nested fns → code becomes more complex → to find a derivative ↓ and to code becomes more difficult.

How is this relevant in case of Deep Learning?



Forward Pass Computation:

1) Linear Transformation:

$$z = w \cdot x + b$$

2) Activation (Sigmoid Function):

$$y_{pred} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

3) Loss Function (Binary Cross-Entropy Loss):

$$L = -[y_{target} \cdot \ln(y_{pred}) + (1 - y_{target}) \cdot \ln(1 - y_{pred})]$$

From here,

with the help of $\hat{y}$ and y → we calculate loss

$\hat{y}$ ↓ prediction, y ↓ input

[To know how our DL is performing]

and then we perform a backward pass (Backward-Propagation)

↳ we need to compute gradients of loss with respect to parameters

$\frac{dL}{dw}$, $\frac{\partial L}{\partial b}$

So, we need to calculate,

$$\frac{\partial L}{\partial y_{pred}} \times \frac{\partial y_{pred}}{\partial z} \frac{\partial z}{\partial w} \rightarrow \frac{\partial L}{\partial w}$$

similarly to get $\frac{\partial L}{\partial b}$, we use above formula and replace $\frac{\partial z}{\partial w}$ to be $\frac{\partial z}{\partial b}$

ALL OF THIS JUST FOR A SINGULAR NEURON INPUT.

↳ Things get extremely difficult when we are handling a deep Neural Network

Autograds - A great tool or feature in LibTorch that provides automatic differentiation capability for tensor operations.

It enables gradient computation, which is essential for training ML models using optimization algos like gradient descent.

Autograd in LibTorch is a C++ memory management system on its own.

→ hidden cam recorder for our math!

Requires to be explicitly turn on, hit play, check footage and erase the tape.

1) The "ON" Switch

The Functions: torch::requires_grad(true) OR .requires_grad_(true)

When to use it? → We only use this on our weights (the variables AI is trying to learn and adjust).

* We never use these on input data (camera images)

```
// By default, tensors do not handle or track gradients
auto x_check = torch::tensor(3.0);
std::cout << x_check.requires_grad() << std::endl;
```

↳ returns 0 (false)

```
// Enable gradient tracking
auto w = torch::tensor(2.0, torch::requires_grad(true));
// OR
auto w2 = torch::tensor(3.0).requires_grad_(true);
```

↓ If we compute all this in maths and compute gradients

```
// y = w * x_check^2 + 3
auto x_val = torch::tensor(3.0);   // x = 3, no gradient needed
auto w_val = torch::tensor(2.0, torch::requires_grad(true));
                                   // w = 2

auto y = w_val * x_val.pow(2) + 3;   // y = 2*(3^2) = 21
std::cout << "y = " << y.item<float>() << std::endl;
                                   // 21

// Compute gradients (backward pass)
y.backward();
// dy/dw = x^2 = 9
std::cout << "dy/dw = " << w_val.grad() << std::endl;
```